Relatório Técnico

Pratical Assignment

Trabalho Prático
Computação Gráfica

Universidade do Minho
Escola de Engenharia, Braga, Portugal
Licenciatura em Engenharia Informática

Ano Letivo 2025/2026

Grupo 25
Carlos Silva; Francisco Veloso; Nuno Soares; Duarte Faria
a106792@uminho.pt
a106882@uminho.pt
a107366@uminho.pt
a95609@uminho.pt

Conteúdo

1 Introdução . . . 4
2 Fase 1: Primitivas e Motor de Visualização . . . 5
2.1 Fluxo de Dados . . . 5
2.2 Generator — Geração de Primitivas . . . 5
2.2.1 Plano (Plane) . . . 5
2.2.2 Algoritmo . . . 5
2.2.3 Caixa (Box) . . . 6
2.2.4 Algoritmo . . . 6
2.2.5 Esfera (Sphere) . . . 6
2.2.6 Algoritmo . . . 7
2.2.7 Cone . . . 7
2.2.8 Geometria do Cone . . . 8
2.2.9 Algoritmo . . . 8
2.3 Engine — Motor Gráfico . . . 8
2.3.1 Formato do Ficheiro .3d . . . 8
2.3.2 Leitura da Configuração XML . . . 9
2.3.3 Pipeline de Renderização . . . 9
2.3.4 Callback de Reshape . . . 9
2.3.5 Callback de Display . . . 9
2.3.6 Sistema de Câmera . . . 10
2.3.7 Câmera Fixa (XML) . . . 10
2.3.8 Câmera Orbital (Explorer) . . . 10
2.3.9 Modos de Renderização . . . 10
2.3.10 Controlos Adicionais . . . 10
2.4 Resultados obtidos . . . 11
2.5 Conclusão . . . 13
3 Fase 2: Transformações Geométricas e Hierarquia de Grupos . . . 14
3.1 Objetivo da Fase 2 . . . 14
3.2 Arquitetura Introduzida . . . 14
3.2.1 Estrutura `Transform` . . . 14
3.2.2 Estrutura `Group` . . . 14
3.3 Parsing XML Recursivo . . . 14
3.3.1 Leitura das Transformações . . . 14
3.3.2 Leitura dos Modelos . . . 14
3.3.3 Leitura de Subgrupos . . . 15
3.4 Renderização Hierárquica . . . 15
3.4.1 Aplicação das Transformações . . . 15
3.5 Validação com Testes da Fase 2 . . . 15
3.6 Resultados Obtidos . . . 16
3.7 Conclusão da Fase 2 . . . 16
4 Fase 3: Curvas de Catmull-Rom e Superfícies de Bezier . . . 17
4.1 Objetivo da Fase 3 . . . 17
4.2 Geração das Superfícies de Bezier . . . 17
4.2.1 Estratégia de Tesselação . . . 17
4.2.2 Formato do Ficheiro Gerado . . . 17
4.3 Curvas de Catmull-Rom no Engine . . . 17
4.3.1 Cálculo da Curva . . . 17
4.3.2 Alinhamento com a Trajetória . . . 17
4.3.3 Visualização da Curva . . . 17
4.4 Parsing XML da Fase 3 . . . 18

4.5 Validação com os Testes da Fase 3 18
4.6 Resultados Obtidos 18
4.7 Conclusão da Fase 3 18

1 Introdução

O presente relatório descreve o desenvolvimento da primeira fase do Trabalho Prático da unidade curricular de Computação Gráfica. O objetivo principal desta fase consistiu na construção da base de um motor gráfico 3D, recorrendo à biblioteca OpenGL/GLUT, e na implementação de um sistema modular composto por duas aplicações distintas: um *generator*, responsável pela criação das primitivas geométricas, e um *engine*, encarregado da leitura de ficheiros de configuração e da renderização da cena.

Ao longo do trabalho, implementámos as várias primitivas tridimensionais, diferentes modos de visualização e um sistema de câmara configurável, cumprindo assim a estrutura fundamental que servirá de base para as fases seguintes do projeto que o nosso grupo implementará.

2 Fase 1: Primitivas e Motor de Visualização

O objetivo da Fase 1 é construir a base do motor 3D que será desenvolvido ao longo das quatro fases do trabalho. Esta fase foca-se em dois componentes fundamentais:

1. **Generator** — aplicação independente que gera ficheiros `.3d` contendo os vértices das primitivas geométricas.
2. **Engine** — aplicação que lê um ficheiro XML de configuração (câmera, janela, modelos) e renderiza a cena com OpenGL/GLUT.

A separação em dois programas distintos permite que a computação pesada da geometria seja feita apenas uma vez.

2.1 Fluxo de Dados

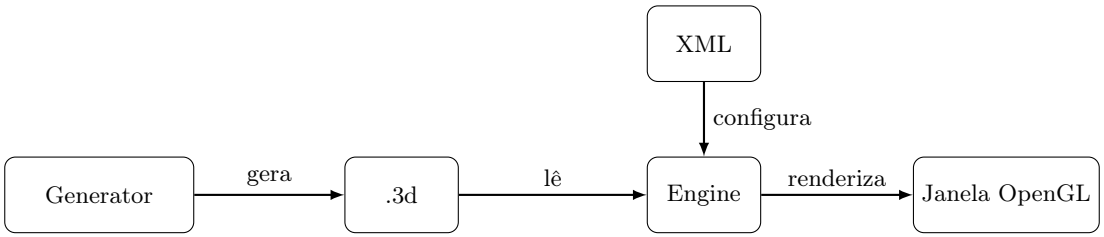


Figura 1: Fluxo de dados entre os componentes.

2.2 Generator — Geração de Primitivas

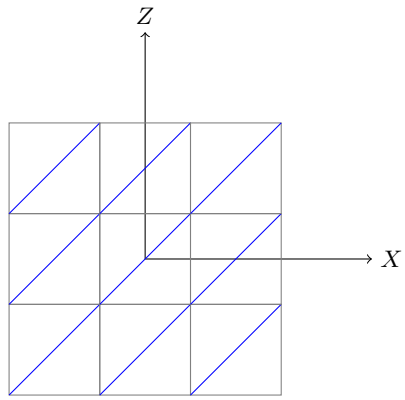
A abordagem adotada no *generator* baseia-se em superfícies paramétricas. Uma decisão fundamental foi o uso do *Counter-Clockwise Winding Order* para todos os triângulos, garantindo que as faces externas sejam corretamente renderizadas pelo OpenGL com *Back-face Culling* ativo. Ao definir os vértices nesta ordem, a face frontal de cada polígono é determinada pela regra da mão direita, permitindo que o *engine* descarte a renderização de faces internas ou não visíveis.

2.2.1 Plano (Plane)

O plano é gerado no plano XZ ($y = 0$), centrado na origem, com um dado comprimento L e número de subdivisões D.

2.2.2 Algoritmo

1. Calcular o tamanho de cada subdivisão: $s = L/D$
2. Calcular o offset de centralização: $h = L/2$
3. Para cada célula (i, j) da grelha $D \times D$:
   - Calcular os 4 cantos: $(x_1, z_1) = (-h + i \cdot s,\ -h + j \cdot s)$ e $(x_2, z_2) = (-h + (i{+}1) \cdot s,\ -h + (j{+}1) \cdot s)$
   - Emitir triângulo 1: $(x_1, 0, z_1),\ (x_1, 0, z_2),\ (x_2, 0, z_2)$
   - Emitir triângulo 2: $(x_1, 0, z_1),\ (x_2, 0, z_2),\ (x_2, 0, z_1)$



Plano com 3 divisões (vista de cima)

Figura 2: Subdivisão do plano. A diagonal azul indica a divisão de cada quad em dois triângulos.

2.2.3 Caixa (Box)

A caixa é centrada na origem com um dado tamanho S e número de divisões D por aresta. Cada uma das 6 faces é uma grelha de $D \times D$ quads, cada quad dividido em 2 triângulos.

2.2.4 Algoritmo

1. Calcular: $h = S/2$ (metade do tamanho), $s = S/D$ (passo da subdivisão)
2. Para cada uma das 6 faces (frente, trás, direita, esquerda, cima, baixo):
   - Para cada célula (i, j) da grelha $D \times D$:
     - Calcular os 4 cantos do quad na face
     - Emitir 2 triângulos com *winding* CCW (visto de fora)

Decisão de projeto: A normal de cada face aponta para fora da caixa. Os triângulos são emitidos com ordem CCW quando vistos do exterior, garantindo compatibilidade com *back-face culling*.

O número total de vértices para a caixa é:

$$V_{\text{box}} = 6 \times D^2 \times 2 \times 3 = 36 \cdot D^2$$

2.2.5 Esfera (Sphere)

A esfera é gerada a partir da conversão de coordenadas esféricas para coordenadas cartesianas. No código, adotámos a convenção em que o ângulo θ representa a componente polar e o ângulo ϕ a componente azimutal.

Equações Paramétricas:

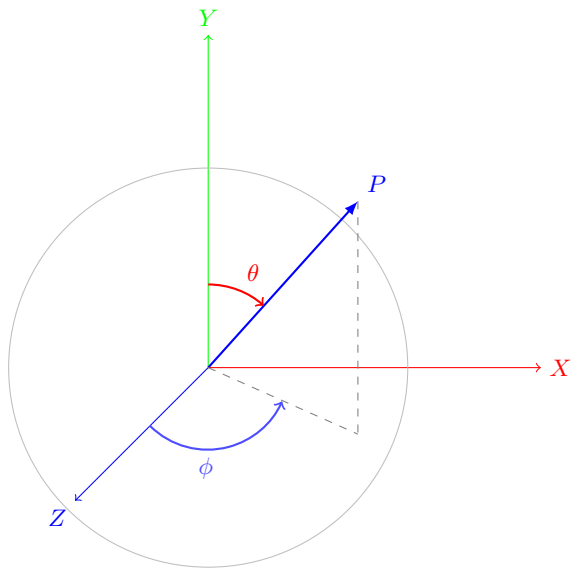
$$\begin{cases} x = r \cdot \sin(\theta) \cdot \sin(\phi) \\ y = r \cdot \cos(\theta) \\ z = r \cdot \sin(\theta) \cdot \cos(\phi) \end{cases} \tag{1}$$

A utilização direta de $\cos(\theta)$ no eixo Y mostra que este é o eixo vertical da esfera, ficando assim alinhado com o sistema de coordenadas padrão do OpenGL.

Durante a implementação, tivemos o cuidado de tratar corretamente os polos da esfera. No topo ($\theta = 0$) e na base ($\theta = \pi$), todos os pontos de cada *slice* coincidem no mesmo local. Se gerássemos quadriláteros nessas zonas (ou seja, dois triângulos por divisão), iríamos obter triângulos degenerados, com área nula.

Para evitar esse problema, o algoritmo identifica as camadas correspondentes aos polos e, nessas situações, gera apenas um triângulo por *slice*, formando um *triangle fan* que converge no polo. Desta forma, reduzimos o número de vértices desnecessários, tornando o processo ligeiramente mais eficiente.

A superfície da esfera é dividida em *slices* S_l (divisões ao longo do ângulo azimutal) e *stacks* S_t (divisões ao longo do ângulo polar). Cada quadrilátero formado entre duas *stacks* e duas *slices* é dividido em dois triângulos através da diagonal que liga o vértice v_1 ao vértice v_3.



θ: ângulo polar, ϕ: ângulo azimutal

Figura 3: Sistema de coordenadas esféricas utilizado na geração da Esfera.

2.2.6 Algoritmo

1. Para cada stack i $(0 \leq i < S_t)$:
   - Calcular $\theta_1 = \pi \cdot i/S_t$ e $\theta_2 = \pi \cdot (i{+}1)/S_t$
2. Para cada slice j $(0 \leq j < S_l)$:
   - Calcular $\phi_1 = 2\pi \cdot j/S_l$ e $\phi_2 = 2\pi \cdot (j{+}1)/S_l$
   - Calcular os 4 vértices: $v_1(\theta_1, \phi_1)$, $v_2(\theta_2, \phi_1)$, $v_3(\theta_2, \phi_2)$, $v_4(\theta_1, \phi_2)$
   - Se $i \neq S_t - 1$: emitir triângulo v_1, v_2, v_3 (não degenera no polo sul)
   - Se $i \neq 0$: emitir triângulo v_1, v_3, v_4 (não degenera no polo norte)

Tratamento dos polos: No polo norte ($\theta = 0$), os vértices v_1 e v_4 coincidem no ponto $(0, r, 0)$, pelo que o triângulo $v_1 v_3 v_4$ degenera e é omitido. No polo sul ($\theta = \pi$), v_2 e v_3 coincidem em $(0, -r, 0)$, pelo que o triângulo $v_1 v_2 v_3$ degenera e é omitido.

O número total de triângulos é:

$$T_{\text{sphere}} = 2 \cdot S_l \cdot (S_t - 2) + 2 \cdot S_l = 2 \cdot S_l \cdot (S_t - 1)$$

2.2.7 Cone

O cone é gerado com a base no plano XZ $(y = 0)$, centrado na origem. O raio diminui linearmente com a altura, de R (na base) a 0 (no vértice).

2.2.8 Geometria do Cone

Para uma stack i ($0 \leq i < S_t$), o raio e a altura são:

$$r_i = R\left(1 - \frac{i}{S_t}\right) \tag{2}$$

$$y_i = h \cdot \frac{i}{S_t} \tag{3}$$

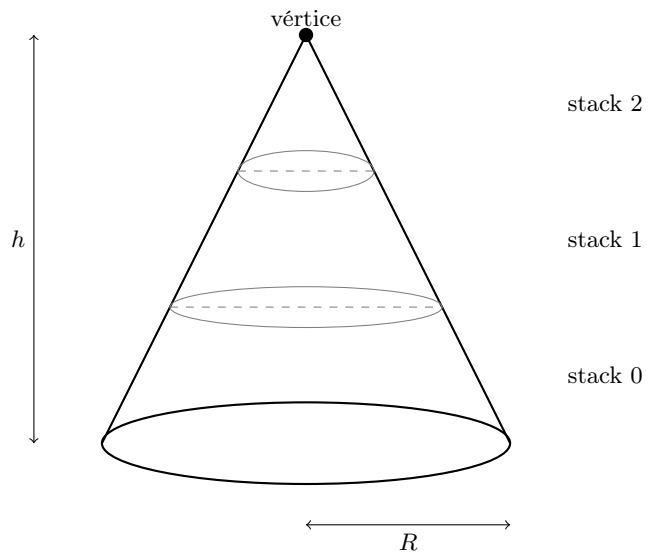


Figura 4: Estrutura do cone com 3 stacks.

2.2.9 Algoritmo

1. **Base** — leque (fan) de triângulos no plano XZ:
   - Para cada *slice* j, é emitido um triângulo com centro em $(0, 0, 0)$ e dois pontos consecutivos da circunferência da base (ordem CCW quando observado de baixo, com a normal orientada segundo $-Y$).
2. **Corpo** — dividido em *stacks*:
   - Para cada *stack* i e *slice* j:
     - Calculam-se os quatro vértices v_1, v_2, v_3, v_4 que definem o *quad*;
     - Emite-se o triângulo v_1, v_3, v_2 (ordem CCW quando observado do exterior);
     - Se $i \neq S_t - 1$ (ou seja, se não for a última *stack*), emite-se também o triângulo v_1, v_4, v_3.

Na última stack, os vértices superiores convergem no vértice do cone, pelo que apenas é emitido um triângulo por slice.

2.3 Engine — Motor Gráfico

2.3.1 Formato do Ficheiro .3d

O formato de ficheiro adotado para armazenar os modelos é simples e textual:

- **Linha 1**: número total de vértices N

– **Linhas 2 a** $N+1$: coordenadas x y z de cada vértice (um por linha)

Os vértices são armazenados em grupos de 3 (formando triângulos), na ordem em que devem ser desenhados com `GL_TRIANGLES`. O *winding order* é **counter-clockwise** (CCW), que é o default do OpenGL para *front faces*.

2.3.2 Leitura da Configuração XML

Usámos a biblioteca *TinyXML-2* para interpretar o ficheiro de cena já que foi a que nos foi recomendada. A decisão de usar XML permite configurar a resolução da janela e os parâmetros da câmara sem necessidade de recompilar o código, garantindo flexibilidade na montagem de diferentes cenários.

O engine recebe como argumento um ficheiro XML que define:

– **Janela**: dimensões (largura, altura)
– **Câmera**: posição, ponto de foco (*lookAt*), vetor *up*, e parâmetros de projeção (FOV, *near*, *far*)
– **Modelos**: lista de ficheiros `.3d` a carregar

Alguns Valores por omissão são definidos para os campos opcionais (`up` $= (0, 1, 0)$, `fov` $= 60$, `near` $= 1$, `far` $= 1000$), conforme especificado na sintaxe do file XML de referência colocado na blackboard.

2.3.3 Pipeline de Renderização

A renderização segue o pipeline clássico do OpenGL com GLUT:

1. **Inicialização**: `glutInit`, configuração do modo de display (RGBA, double buffer, depth buffer), criação da janela.
2. **Configuração OpenGL**: ativação do *depth test* e *back-face culling*.
3. **Registo de callbacks**: display, reshape, keyboard.
4. **Ciclo principal**: `glutMainLoop`.

2.3.4 Callback de Reshape

No redimensionamento da janela:

1. Ativar a matriz de projeção
2. Resetar a matriz
3. Configurar o *viewport* e aplicar a projeção perspetiva com os parâmetros do XML
4. Voltar à matriz de *modelview*

2.3.5 Callback de Display

A cada frame:

1. Limpar buffers de cor e profundidade
2. Posicionar a câmera com `gluLookAt`
3. Definir o modo de polígono (*solid*/*wireframe*/*points*)
4. Desenhar eixos de referência (com *depth test* ativo)
5. Desenhar os modelos com triângulos (modo imediato)
6. Trocar buffers (*double buffering*)

Nota: Desenhámos os eixos **antes** dos modelos, mantendo o *depth test* ativo em ambos os casos. Isto garante que a geometria dos modelos que está mais próxima da câmara se sobrepõe naturalmente aos eixos, o que respeita corretamente a profundidade.

Inicialmente, não tínhamos implementado desta forma e encontrámos alguns problemas na renderização das figuras. Embora a figura fosse estruturalmente igual à dos testes, algumas linhas que evidenciavam a profundidade não eram renderizadas corretamente.

2.3.6 Sistema de Câmera

Foram implementados dois modos de câmera:

2.3.7 Câmera Fixa (XML)

Usa diretamente os valores de posição, *lookAt* e *up* definidos no ficheiro XML.

2.3.8 Câmera Orbital (Explorer)

Implementou-se uma câmara orbital baseada em coordenadas esféricas para facilitar a inspeção dos modelos. O utilizador controla os ângulos α (rotação horizontal) e β (inclinação vertical).

Algorithm 1 Conversão para Câmara Orbital

1: $pos.x \leftarrow raio \cdot \cos(\beta) \cdot \sin(\alpha)$
2: $pos.y \leftarrow raio \cdot \sin(\beta)$
3: $pos.z \leftarrow raio \cdot \cos(\beta) \cdot \cos(\alpha)$

onde α é o ângulo azimutal, β é a elevação (limitada a ± 1.5 rad $\approx \pm 86$ para evitar *gimbal lock*), e r é a distância ao foco (Q/E).

A câmera orbital é inicializada a partir da posição XML, convertendo coordenadas cartesianas para esféricas:

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2} \tag{4}$$

$$\alpha = \text{atan2}(p_x,\ p_z) \tag{5}$$

$$\beta = \arcsin\left(\frac{p_y}{r}\right) \tag{6}$$

2.3.9 Modos de Renderização

Três modos são suportados:

Tecla	Modo	Descrição
1	Sólido	Faces preenchidas
2	Wireframe	Apenas arestas
3	Pontos	Apenas vértices

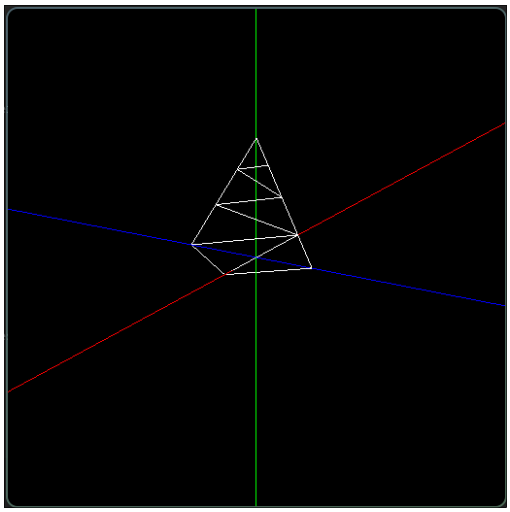
Tabela 1: Modos de renderização disponíveis.

2.3.10 Controlos Adicionais

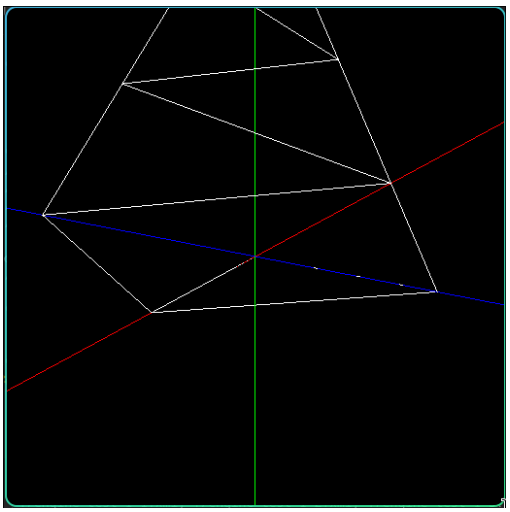
Tecla	Ação
W/S	Rodar câmera verticalmente (elevação)
A/D	Rodar câmera horizontalmente (azimute)
Q/E	Zoom in/out (alterar raio)
C	Alternar câmera fixa/orbital
ESC	Sair da aplicação

Tabela 2: Controlos do engine.

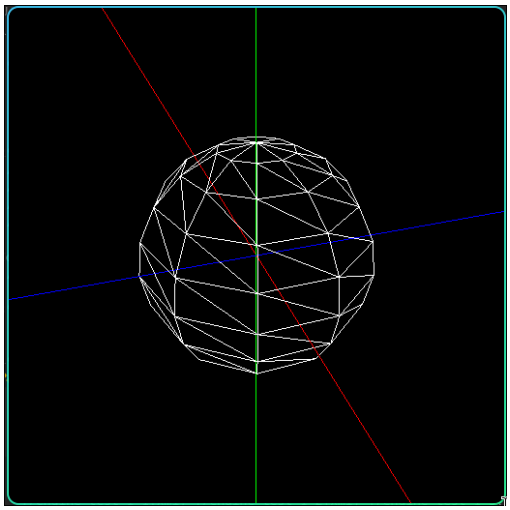
2.4 Resultados obtidos



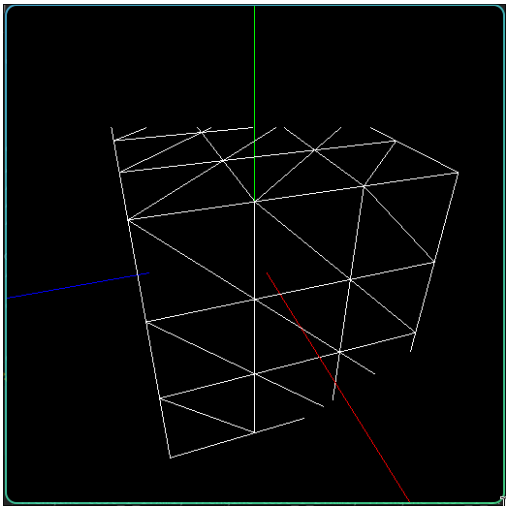
(a) Teste 1_1



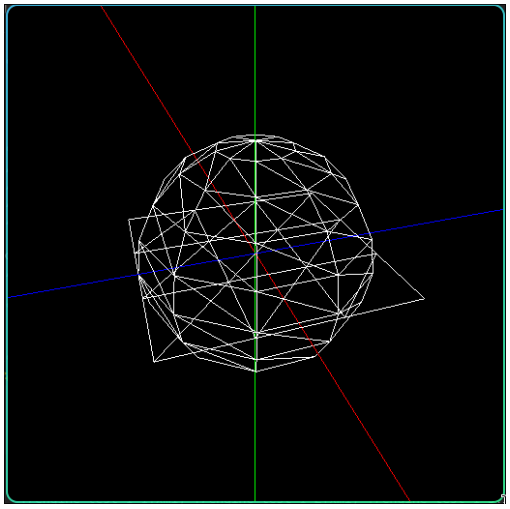
(b) Teste 1_2



(c) Teste 1_3



(d) Teste 1_4



(e) Teste 1_5

Figura 5: Resultados dos testes

2.5 Conclusão

Nesta primeira fase, implementámos corretamente as quatro primitivas que nos foram solicitadas no generator, bem como o engine capaz de ler as configurações em XML e renderizar os modelos com OpenGL/GLUT, tal como praticámos nas aulas. O nosso projeto inclui algumas configurações de câmaras, como a fixa e a orbital, três modos de renderização e os respetivos eixos de referência.

De uma forma geral, o nosso grupo sente que conseguiu realizar, sem grandes dificuldades, o que nos foi proposto para esta primeira fase de entrega. O único detalhe que demorou um pouco mais a ser concretizado foi a confirmação e correção da imagem produzida pelo nosso engine, de modo a que fosse exatamente igual à fornecida na diretoria com as imagens dos resultados esperados. A parte de gerar as figuras decorreu sem problemas. No entanto, garantir que ficassem exatamente iguais às imagens de teste exigiu mais algum tempo.

3 Fase 2: Transformações Geométricas e Hierarquia de Grupos

3.1 Objetivo da Fase 2

O objetivo principal desta fase foi evoluirmos o *engine* de uma renderização plana (lista simples de modelos) para uma renderização hierárquica com transformações geométricas, de acordo com a sintaxe XML proposta no enunciado. Em termos práticos, os requisitos implementados foram:

- suporte às transformações `translate`, `rotate` e `scale`;
- suporte a grupos aninhados (*scene graph*);
- aplicação recursiva de transformações com herança pai-filho;
- preservação da ordem das transformações conforme definida no XML.

3.2 Arquitetura Introduzida

Para suportar a nova organização da cena, introduzimos dois novos tipos de dados no diretório `engine/scene`:

- `Transform`: representa uma transformação individual (tipo + parâmetros);
- `Group`: representa um nó da cena, que contém uma lista de transformações, modelos e grupos filhos.

Esta decisão permitiu separar claramente:

1. a descrição da cena (XML),
2. a estrutura em memória (*scene graph*),
3. a renderização recursiva em OpenGL.

3.2.1 Estrutura `Transform` :

Cada transformação é armazenada de forma genérica da seguinte forma:

- tipo (`Translate`, `Rotate`, `Scale`),
- componentes `x,y,z`,
- ângulo (que é usado apenas em `Rotate`).

3.2.2 Estrutura `Group` :

Cada grupo contém:

- `vector<Transform>`, que são as transformações locais do grupo;
- `vector<string>`, são os ficheiros `.3d` desse grupo;
- `vector<Group>`, são os filhos para a renderização recursiva.

Desta maneira, qualquer subgrupo herda automaticamente as transformações acumuladas do seu pai.

3.3 Parsing XML Recursivo

Na Fase 1 o parser carregava apenas uma lista linear de modelos. Na Fase 2 passámos a construir uma árvore de grupos através de uma função recursiva `parseGroup(XMLElement*)`.

3.3.1 Leitura das Transformações As transformações são lidas a partir do nó `<transform>` e inseridas no vetor pela mesma ordem em que surgem no XML. Esta opção é crítica porque transformação geométrica não é comutativa; por exemplo, *translate seguido de rotate* produz resultado diferente de *rotate seguido de translate*.

3.3.2 Leitura dos Modelos Cada nó `<models>` pode conter vários `<model file="...»`. Os nomes dos ficheiros são guardados no grupo corrente.

3.3.3 Leitura de Subgrupos

Cada `<group>` pode conter outros `<group>` e a função recursiva cria essa hierarquia em memória. O resultado final é armazenado em `WorldConfig::rootGroup`.

3.4 Renderização Hierárquica

No *engine*, a renderização passou a ser feita por uma função recursiva `renderGroup(const Group&)`:

1. `glPushMatrix()` para guardar a matriz atual;
2. aplicar as transformações locais do grupo;
3. desenhar os modelos desse grupo;
4. renderizar recursivamente os filhos;
5. `glPopMatrix()` para restaurar a matriz anterior.

Ou seja, um modelo filho é desenhado no referencial já transformado do pai.

3.4.1 Aplicação das Transformações :

Implementámos uma função auxiliar `applyTransform` com um mapeamento direto para facilitar a implementação posteriormente:

- `Translate` → `glTranslatef(x,y,z)`
- `Rotate` → `glRotatef(angle,x,y,z)`
- `Scale` → `glScalef(x,y,z)`

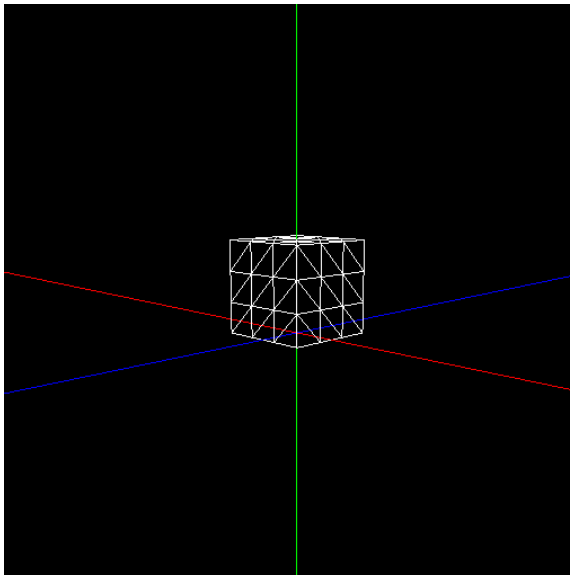
3.5 Validação com Testes da Fase 2

A validação foi feita com os quatro ficheiros disponibilizados para a Fase 2 fornecidos pelos professores:

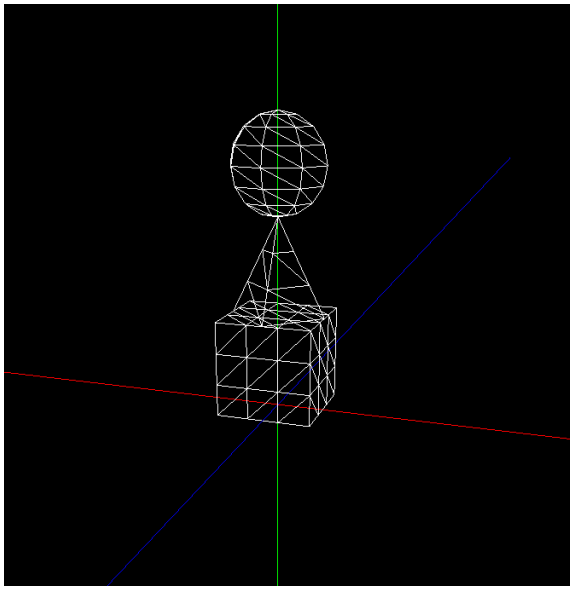
- `test_2_1.xml`, que é uma transformação simples;
- `test_2_2.xml`, uma hierarquia pai-filho-filho;
- `test_2_3.xml`, uma composição de transformações;
- `test_2_4.xml`, são múltiplas Box's em torno do centro com rotações diferentes.

Durante esta fase o nosso grupo possuiu alguns problemas nos testes de em que possuíamos alguma box, visto que a maneira que estava a ser gerada algumas faces tornava a geração dos triângulos diferente da imagem de teste fornecida pelos professores.

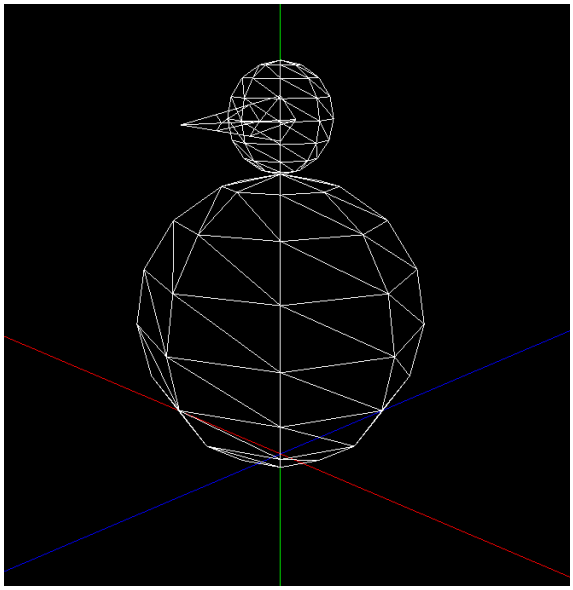
3.6 Resultados Obtidos



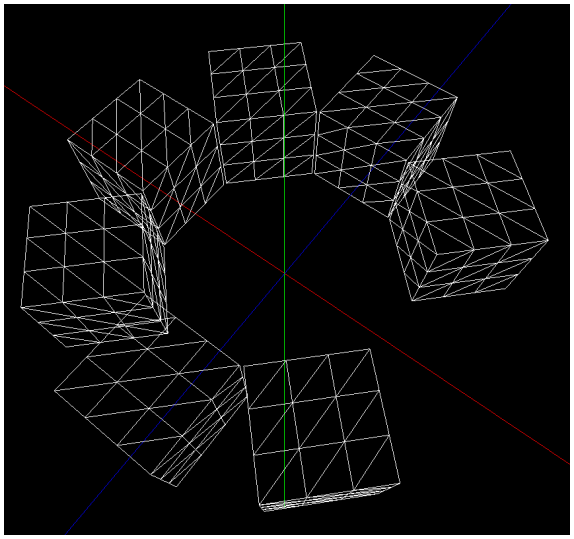
(a) Teste 2_1



(b) Teste 2_2



(c) Teste 2_3



(d) Teste 2_4

Figura 6: Resultados visuais da Fase 2

3.7 Conclusão da Fase 2

A Fase 2 ficou concluída com a introdução de transformações geométricas estáticas e estrutura hierárquica de grupos no *engine*. A principal diferença foi arquitetural (como já tinha sido mencionado no enunciado). Passámos de uma cena plana para uma *scene graph* recursiva, preservando a ordem de transformações e garantindo herança entre níveis.

4 Fase 3: Curvas de Catmull-Rom e Superfícies de Bezier

4.1 Objetivo da Fase 3

Nesta fase o objetivo foi acrescentar movimento ao modelo através de curvas de Catmull-Rom e introduzir a geração de superfícies de Bezier a partir de ficheiros `.patch`. Em termos funcionais, esta fase exigiu duas partes distintas:

- geração de primitivas Bezier no *generator*, da mesma maneira que foram feitas as anteriores;
- suporte no *engine* para `translate` animado com pontos de controlo e orientação pela tangente da curva.

4.2 Geração das Superfícies de Bezier

No *generator*, a implementação passou a ler o ficheiro `.patch`, e de seguida interpretar os índices dos pontos de controlo e calcular os vértices da superfície através das bases de Bernstein. Cada patch cúbico é subdividido em quadrados paramétricos e depois convertido em triângulos para formar o ficheiro `.3d` final.

A lógica usada segue a fórmula clássica de Bezier bidimensional:

$$P(u,v) = \sum_{i=0}^{3} \sum_{j=0}^{3} B_i(u) B_j(v) P_{ij}$$

onde B_i representam os polinómios de Bernstein e P_{ij} os pontos de controlo do patch.

4.2.1 Estratégia de Tesselação A tesselação foi parametrizada por um valor inteiro recebido na linha de comandos. Quanto maior o valor, maior o número de triângulos gerados e mais suave fica a superfície final. Para a fase entregue foi usado `tess = 10`.

4.2.2 Formato do Ficheiro Gerado O ficheiro gerado pelo nosso *generator* mantém o formato usado nas fases anteriores, começando com o número total de vértices e listando depois os pontos em sequência. Isso permitiu reutilizar a lógica de carregamento no *engine* sem introduzir alterações estruturais no leitor de modelos.

4.3 Curvas de Catmull-Rom no Engine

No *engine*, a principal alteração nesta fase 3 foi o suporte a uma `translate` temporal, definida por um conjunto de pontos de controlo no XML. Em vez de uma translação fixa, o modelo passa a deslocar-se ao longo de uma curva de Catmull-Rom fechada, calculada a partir desses pontos.

Para cada instante da animação, o *engine* calcula:

- a posição atual na curva;
- a derivada da curva para obter a direção do movimento;
- a matriz de orientação quando a opção `align` está ativa.

4.3.1 Cálculo da Curva O cálculo da curva foi implementado com a fórmula de Catmull-Rom, usando quatro pontos consecutivos para cada segmento.

4.3.2 Alinhamento com a Trajetória Quando o atributo `align` é ativado, o objeto não só se desloca ao longo da curva como também roda para acompanhar a direção tangente. Para isso foi construída uma matriz ortonormal a partir da derivada, usando um vetor *up* fixo como referência.

4.3.3 Visualização da Curva Para facilitar a validação visual e o resultado obtido ser igual ao dos testes fornecidos pelos professores, a trajetória é desenhada no ecrã em simultâneo com o movimento do objeto. Esta linha ajuda a confirmar se a trajetória e a posição do modelo estão coerentes com os pontos de controlo definidos no XML (inicialmente não era desenhada a linha).

4.4 Parsing XML da Fase 3

O parser foi estendido para reconhecer novas variantes de `translate` e `rotate`:

- `translate time="..."` com pontos de controlo;
- atributo `align` para ativar orientação pela trajetória;
- `rotate time="..."` para rotações animadas.

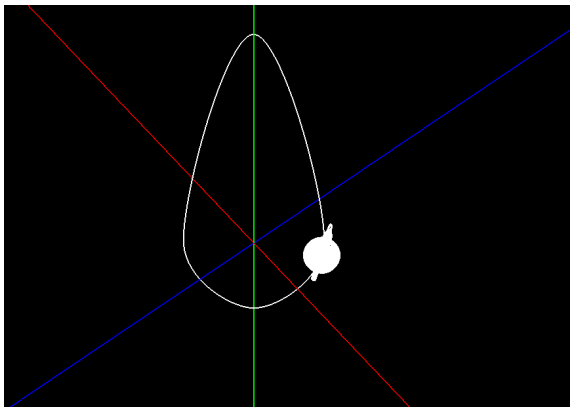
Esta alteração manteve a compatibilidade com as transformações estáticas já usadas na Fase 2. Caso a translação não tenha tempo associado, o comportamento continua a ser uma translação fixa normal, ou seja funciona e é compatível até com as fases anteriores.

4.5 Validação com os Testes da Fase 3

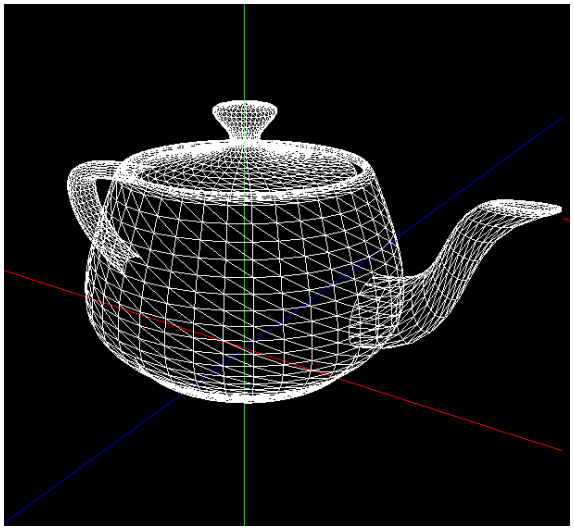
A validação foi feita com os testes disponibilizados para a fase 3:

- `test_3_1.xml`, com movimento animado ao longo de uma curva de Catmull-Rom;
- `test_3_2.xml`, com outra configuração de animação e orientação pela trajetória.

4.6 Resultados Obtidos



(a) Teste 3_1



(b) Teste 3_2

Figura 7: Resultados visuais da Fase 3

4.7 Conclusão da Fase 3

Durante o desenvolvimento desta fase, surgiram vários obstáculos que exigiram investigação e ajustes no código. A primeira dificuldade foi a integração das structs como `Point3D` e `vector<Point3D>` nos campos das transformações. Isto obrigou-nos a reorganizar os includes dos headers e a garantir compatibilidade com o sistema de tipos já existente qe tinhamos, particularmente quando se cruzava o parser com os novos tipos.

Outro desafio importante foi garantir que os modelos fossem carregados para GPU apenas após a criação do contexto OpenGL. Inicialmente, a nossa lógica de preload estava a ser chamada demasiado cedo, o que nos estava a causar erros.

Por fim, manter a compatibilidade com as transformações estáticas da Fase 2 e garantir que os testes se mantinham inalterados enquanto se adicionava suporte a estas novas transformações temporais exigiu que tivéssemos cuidado na estruturação de condicionais e fallbacks. Inicialmente, pensámos em separar a lógica desta fase para uma função diferente, de modo a torná-la mais modular, mas decidimos mantê-la na mesma parte por ser mais intuitivo.

Referências

1. Não foram utilizadas referências bibliográficas externas. O trabalho foi desenvolvido com base nos slides teóricos, práticos e os guiões lecionados na unidade curricular.